

Projet Informatique Graphique 3

Physically Based Rendering et Animation

Oscar Gledel

March 19, 2026



Fig. 1. Aperçu d'un résultat visuel impliquant l'ensemble des techniques présentées dans ce rapport comme le chargement de modèles, l'animation (robots), le PBR, ou encore les reflets de la Skymap HDR (casque).

Ce rapport présente le travail effectué dans le cadre d'un projet d'informatique graphique. Il contient une implémentation du rendu physiquement réaliste (PBR), un système d'animation ainsi que des Skybox en High Dynamic Range (HDR). Le code n'est pas explicitement présenté, sauf dans les cas où l'algorithmique est intéressante, mais l'ensemble du projet est disponible sur la page Github dont le lien se trouve en bas de chacune des pages.

Le projet a pour but de reprendre le travail effectué en Master 1 dans l'UE Informatique Graphique 1, afin d'y ajouter de nouveaux éléments. En particulier, il était demandé d'implémenter les matériaux de type PBR (Physically Based Rendering) ainsi qu'un élément de rendu parmi une liste fournie sur la page Moodle de l'UE. En parallèle, il était également demandé d'implémenter un système d'animation afin de pouvoir au minimum animer et déformer un cylindre.

Dans la suite, je détaillerai dans un premier temps la partie animation et son implémentation, abordant en particulier le système de squelette et les animations par keyframes interpolées. Dans une seconde partie, je présenterai mon implémentation du rendu physiquement réaliste, ou Physically Based Rendering (PBR) et les difficultés rencontrées. Dans une troisième partie, j'aborderai la gestion des Cubemaps/Skybox et notamment celles au format HDR (pour High Dynamic Range) qui demandent une attention particulière. Dans une dernière partie sur le rendu, je détaillerai ma gestion des maillages pour permettre d'en charger plusieurs sans ralentissement, et j'en profiterai pour aborder quelques éléments divers qui ont à la fois servi à simplifier l'utilisation de mon outil et à l'optimiser. En conclusion, je ferai un bilan du temps de travail accordé au projet et développerai quelques améliorations et optimisations que je souhaite apporter à mon projet dans le futur.

1. Animations

Le système d'animation que j'ai implémenté est celui du site LearnOpenGL. J'utilise donc l'approche Dual-Quaternions qui évite les torsions/déformations du maillage. En termes d'implémentation, la matrice des transformations de chaque os est calculée dans le code C++ (côté CPU) puis envoyée au vertex shader en même temps que les poids des 4 os les

plus importants pour le vertex. La transformation est alors calculée dans le vertex shader (côté GPU) puis les différentes informations (position, normale, tangente) sont envoyées au fragment shader pour l'affichage final.

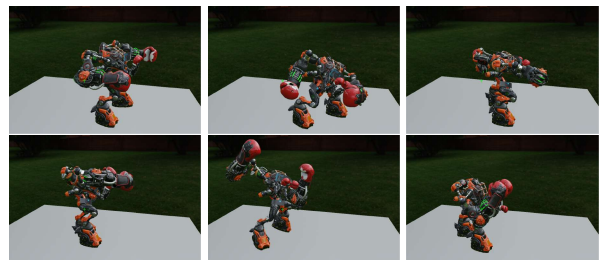


Fig. 2. Keyframes de l'animation du robot prises en exemple du mouvement effectué en animation par Dual Quaternions.

Pour ce qui est de l'animation en elle-même, j'utilise un système de keyframes qui me permet d'interpoler différentes positions présentes dans les fichiers d'animations afin de lisser l'évolution des os et ainsi donner l'impression d'un mouvement fluide avec un nombre réduit d'informations. La Figure 2 montre quelques frames d'une animation complexe sur un maillage de robot boxeur.

2. Physically Based Rendering (PBR)

A. Matériaux PBR. L'implémentation de mes matériaux physiquement réalistes (PBR) suit la spécification glTF. Elle comporte ainsi notamment des fonctions telles que `FresnelSchlick()`, `GeometrySmith()` ou encore `GeometrySchlickGGX()`. La Figure 3 montre quelques exemples de matériaux rendus avec du PBR.

J'ai également ajouté la lecture des cartes de textures (albedo, roughness, metallic, etc.) pour obtenir des rendus de matériaux plus fins et réalistes, comme le montre à nouveau la Figure 3.

B. Parallax mapping. J'ai choisi le parallax mapping plutôt que la Tessellation car mon ordinateur sans carte graphique n'aurait pas supporté l'augmentation significative du nombre de triangles. Le parallax mapping que j'ai implémenté s'appuie

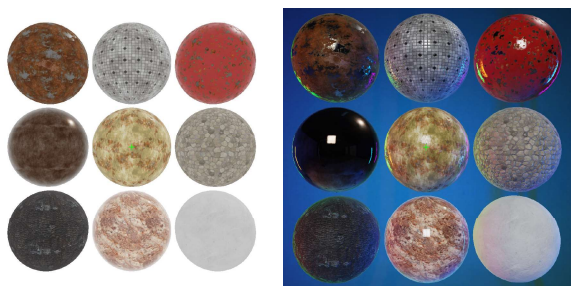


Fig. 3. Quelques exemples du rendu des matériaux PBR dans une scène à fond blanc (à gauche) et dans une scène éclairée par des sources extérieures (à droite) (cf. Irradiance Map).

sur l'idée que, selon l'angle de la caméra et la carte de hauteur (height map), on peut décaler les indices UV d'un fragment pour donner l'impression que la texture s'étire. Quand elle s'étire dans les zones qui sont censées être plus élevées, on peut percevoir cet aspect "élévation" sans pour autant ajouter de détail en termes de nombre de triangles.

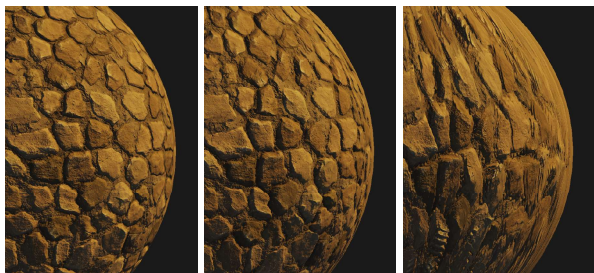


Fig. 4. Application de différents niveaux de parallax sur la texture. De gauche à droite, le facteur de distorsion est de 0, 0.045, 0.3.

La Figure 4 montre ce que produit cet effet. Il est important de noter, comme le montre la figure, qu'il faut utiliser le facteur de distorsion avec parcimonie, car le décalage d'UVs peut rapidement créer des ruptures très visibles dans la texture.

C. Shadow map. Pour l'implémentation des shadow maps, j'ai appliqué le principe du cours qui consiste à évaluer la depth map du point de vue de la lumière avant la passe de rendu PBR. Ensuite, on effectue un changement de repère du fragment dans le shader principal pour connaître sa profondeur dans le repère de la lumière. Quand la valeur est supérieure à la valeur dans la depth map, le pixel est dans l'ombre.

Les shadow maps posent deux problèmes. Le premier, c'est qu'elles sont soumises à la résolution du buffer dans lequel elles sont écrites. Ainsi, on peut voir apparaître, si la résolution est trop basse ou le point de vue trop large, les pixels de la carte dans le rendu des ombres (cf. Figure 5). Le second problème, c'est que ce type de carte demande une passe de rendu simple par lumière directionnelle (Spot ou Directional light), et 6 passes dans le cas d'une spot light. Le coût en mémoire devient rapidement élevé. En raison des limitations de ma machine, j'ai choisi de n'appliquer que les ombres de la lumière directionnelle servant de soleil dans ma scène.

J'ai également implémenté, comme le montre la Figure 5, le Percentage Closest Filtering (PCF) dans mes shadow maps. Cela permet de lisser progressivement l'intensité de l'ombre, et ainsi d'éviter les cassures.



Fig. 5. Mise en évidence de l'effet de la résolution de la shadow map. Ici, la distance de la prise de vue est différente mais la résolution de la carte reste fixe (1024 pixels). Les distances sont, de gauche à droite, 2, 10 et 50.

3. Cubemap & High Dynamic Range (HDR)

A. Cubemaps simples. Les Cubemaps sont des images que l'on projette sous forme de textures sur les faces intérieures d'un cube, de manière à donner l'impression à l'utilisateur qu'il est dans un espace libre grand et infini. Pour cela, on récupère 6 textures carrées (que l'on peut aussi récupérer sous une seule texture qui contient 4 images) et on effectue une première passe de rendu dans un shader de ces textures sur les faces d'un cube, en désactivant le depth buffer. Cela va avoir pour effet de donner l'impression que la cubemap est infiniment loin et qu'il s'agit du décor. Visuellement, une fois un objet dessiné, on obtient le rendu de la Figure 6.

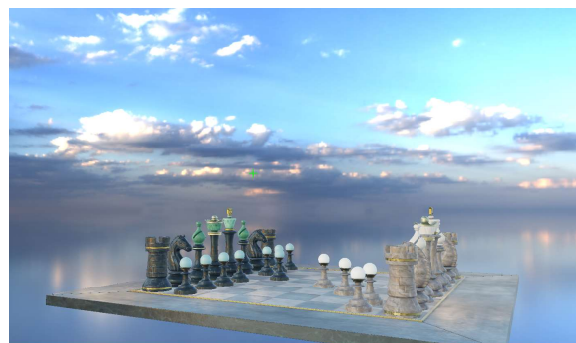


Fig. 6. Rendu de la cubemap (en fond) effectué en désactivant le Z-Buffer, donnant ainsi l'impression que la scène s'étend à l'infini par rapport au jeu d'échecs.

B. Cubemaps HDR. Pour les cubemaps HDR, il faut transformer un fichier .hdr en une cubemap affichable, tout en préservant le principe du HDR qui permet de passer à des valeurs de luminance supérieures à 1 (ou 255 en niveaux de gris).

Le processus est donc le suivant : Lire l'image en .hdr et la dessiner dans un cube pour récupérer 6 buffers images qui constitueront notre environment map finale. Dans un second temps, on traite cette environment map en appliquant un flou pondéré par l'intensité, afin d'en extraire une carte de radiance. Cette carte sert ainsi à l'éclairage global. Une Cubemap HDR est donc composée de deux SamplerCube : environmentMap et irradianceMap. La Figure 7 montre l'effet que produit l'utilisation de ces deux cartes dans l'éclairage ambiant et les reflets des différents matériaux.

C. Tone mapping & Gamma correction. Dans ce projet, j'ai utilisé la méthode de tone mapping ACES (Acamedy Color Encoding System) qui est celle présente dans le moteur Unreal



Fig. 7. Casque sci-fi éclairé par l'irradiance map et réfléchissant l'environnement map d'un ciel éclairé par le jour (à gauche) et au milieu d'une nuit étoilée (à droite). On voit que le niveau de luminosité varie à certains endroits du casque et surtout entre les deux cubemaps, mettant ainsi en évidence le rôle de l'irradiance map et du HDR.

Engine 5, ainsi qu'une gamma correction standard avec un coefficient à 2.2. La figure [Figure 8](#) montre l'impact de ces deux corrections de couleurs en fin du pipeline de rendu.

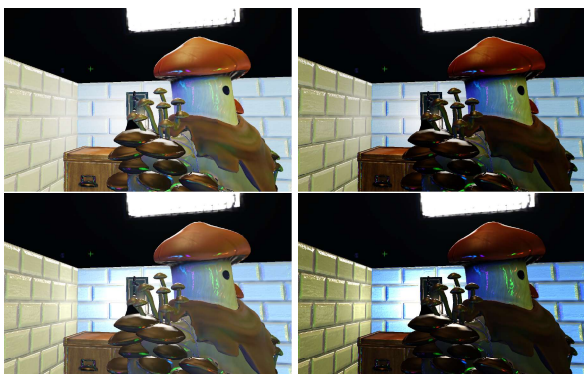


Fig. 8. Impact de la présence/absence de tone mapping ACES et de gamma correction sur le rendu de la scène. La première ligne présente les images avec Tone mapping, la seconde sans. Pour les colonnes, la première présente les images avec Gamma correction, la seconde sans.

On peut voir que c'est la correction gamma qui a le plus d'impact sur la qualité du rendu. Ceci est dû au fait que la couleur albedo des matériaux n'est pas définie dans un espace linéaire, mais en sRGB. En recalant la couleur avec la gamma correction, on améliore beaucoup le résultat. Pour ce qui est du tone mapping, il permet surtout d'éviter les zones de forte intensité lumineuse, comme au niveau des sources ponctuelles.

4. Gestion des données et autres améliorations

A. Gestion des objets. Pour permettre le chargement de multiples objets identiques, j'ai implémenté un système de Géométrie séparée des objets (comme dans certains moteurs de jeux/rendus type Unreal Engine). Le principe est que chaque objet possède un pointeur vers la géométrie qu'il représente, mais cette géométrie n'est présente qu'une fois en mémoire.

Ainsi, elle n'est chargée que dans un seul VAO/VBO et donc envoyée une seule fois à la carte graphique. Les objets identiques encapsulent donc seulement une transformation de ce maillage, et celui-ci est répliqué pour chaque objet à des positions différentes.

B. Gestionnaires de ressources. Toujours dans un objectif d'économie de mémoire et d'optimisation, l'ensemble des ressources chargées dans mon projet (objets, textures, cube-maps, etc.) passent par des Gestionnaires de ressources (Resource managers). Ces classes singletons permettent de ne

charger chaque ressource qu'une seule fois, et ensuite de simplement renvoyer un pointeur quand on en a besoin. Ceci est fait de manière totalement opaque pour l'utilisateur (moi-même en l'occurrence), ce qui simplifie grandement le code que j'ai pu écrire par la suite.

C. Interface ImGui & Scènes. Pour éviter de recompiler le projet de nombreuses fois, j'ai développé un mini-programme à l'intérieur de mon moteur capable de parser des fichiers JSON décrivant une scène. Je peux donc éditer des scènes complexes en amont, choisir les modèles, et surtout en avoir plusieurs, et ensuite les lire au moment de l'exécution sans avoir à recompiler.



Fig. 9. Aperçu de l'interface ImGui du projet. Les différents menus hiérarchisés permettent de modifier les propriétés des matériaux, objets, lumières et cubemaps sans recompiler ou même relancer le programme.

J'ai également ajouté une interface ImGui à mon projet. Cette interface contient de nombreux éléments pour éditer la scène en temps réel, ainsi qu'un affichage du nombre de FPS, comme le montre la [Figure 9](#). L'avantage de cette interface, c'est qu'elle est construite dans sa partie inférieure, de manière à afficher tous les objets quel que soit leur nombre. J'utilise une structure arborescente dans ma scène, avec des systèmes de Nodes, ce qui me permet d'appeler récursivement l'affichage de mon interface en fonction de la structure de ma scène.

5. Conclusion

A. Temps de travail. En termes de temps de travail sur ce projet, la quantité attendue dans le cadre du module était de **46 heures**. Dans mon cas, j'y ai passé **80 heures**, comme le montre la [Table 1](#).

De mon point de vue, le temps accordé au projet me semble correct au vu du résultat. La majorité des notions m'étaient inconnues avant le démarrage du projet, ce qui implique que j'ai dû les prendre en main puis les implémenter. De plus, ayant amélioré mon projet entre le M1 et le M2, je devais intégrer les nouvelles fonctionnalités dans une architecture qui commençait à être complexe. Avec du recul cependant, je pense qu'en se concentrant sur les fonctionnalités essentielles et en passant au-dessus de certaines étapes d'architecture ou d'optimisation (qui ne sont pas forcément nécessaires pour un projet comme celui-ci), les 46 heures sont un temps adéquat.

B. Améliorations futures. Pour ce qui est des améliorations futures, elles sont nombreuses, mais en voici quelques-unes qui me semblent les plus prioritaires.

Implémenter l'anti-aliasing : actuellement, dans certaines scènes, les surfaces vues de manière trop rasante donnent un effet de très mauvaise qualité au rendu. L'anti-aliasing permettrait de corriger cela.

Optimiser le chargement des données : un des goulots d'étranglement principaux en termes de performance, c'est le temps de chargement des ressources au démarrage du programme. J'aimerais implémenter des optimisations sur la lecture des textures, mais également permettre de modifier des textures ou d'ajouter des objets sans relancer le programme.

Passer en Deferred Shading : pour mieux contrôler mon pipeline de Rendu, je voudrais passer au deferred shading qui permet plus de liberté et est un standard de l'industrie aujourd'hui (ainsi qu'un bon moyen de me perfectionner dans la compréhension du pipeline de rendu).

Finalement, j'aimerais reprendre ce projet du début, maintenant que j'ai une bien meilleure compréhension du fonctionnement du rendu temps réel. Si je décide de faire cela, je pense que j'en profiterai pour apprendre Vulkan qui devient progressivement la norme pour le rendu aujourd'hui.

Tâche	Temps (h)
1. Animation	
Lecture de l'article LearnOpenGL	2
Implémentation de l'article LearnOpenGL	2
Adaptation du code à l'architecture	4
Correction de bugs sur l'animation simple	2
Lecture de fichiers .glb (Animation + PBR)	2
Correction de bugs lecture des .glb	2
2. Physically Based Rendering	
Restructuration des classes matériaux du projet M1	4
Lecture de la spécification glTF	2
Implémentation des shaders	5
Correction de bugs sur les shaders	5
Tests sur divers matériaux	1
Shadow maps	2
Parallax mapping	2
3. Cubemap & HDR	
Lecture de l'article Cubemap LearnOpengl	1
Implémentation de la Cubemap	2
Intégration au système existant	2
Lecture des articles sur les Cubemaps HDR	2
Implémentation de la transformation des fichiers HDR	4
Réflexion de la Cubemap sur les matériaux PBR	2
4. Maillages complexes	
Passage à la librairie assimp au lieu du parser custom	2
Système de hiérarchie par nodes	2
Optimisation par pointeurs sur les géométries	4
Intégration des cartes de textures	2
5. Divers	
Interface ImGui pour la gestion scène	3
Refonte de l'architecture du projet & gestion bugs	10
Chargement de fichiers .json pour la lecture des scènes	5
Création de managers de ressources	4
Total des heures	80

Table 1. Répartition du temps de travail estimé par section du projet